

Collaborative Doc For Test 5 Studying

(Montazeri 01 and Davis 02 - Spring 2024)

We have created a document for students to collaborate on ideas for approaching problems for test 5. The aim is **to share ideas** while reducing redundant questions on the piazza. **I will, and maybe other teaching staff, though it is not required as I am volunteering my own time (i.e., do not bother the others), respond when I can. Ideally, students will help each other :)**

Paste the question under the correct category and question number, along with some initial ideas for us to work off. I will likely not respond to questions with no initial idea, as that's pretty much spoon-feeding.

Required ground rules:

1. Do not delete other people's comments.
2. Do not use any highlighting.
3. Do not share code or answers. You can say procedurally what you would like to do. However, do not give explicit code.
 - a. Example of Good Response:
 - i. Keep an integer array (let's call it `set_arr`), that has size as $\max(A)$. Set everything in `set_arr` to 0. Then for every number, $A[i]$, add one to `set_arr[A[i]]`. Then go through `set_arr[A[i]]`, and find the 1.
 - b. bad (sorry whoever I took this from):
 - i. So let's call the end number $A[\text{end}]$ and the beginning number $A[\text{beg}]$.
 - ii. If $A[\text{end}] + A[\text{beg}]$ is x , you return.
 - iii. If $A[\text{end}] + A[\text{beg}]$ is greater than x , then $\text{end} = (\text{end} + \text{beg})/2$.
 - iv. If $A[\text{end}] + A[\text{beg}]$ is less than then $\text{beg} = (\text{end} + \text{beg})/2$.
4. Be appropriate. This means not posting anything inappropriate, such as posting non-relevant materials or purposefully writing incorrect ideas.

Anyone who breaks the ground rules will have their changes reverted and blocked from this and all future collaborative documents.

Legend	
Yellow Highlighted Text	A Course Staff Response.
Red Highlighted Solution	A course staff believes this response is not correct.
Orange Highlighted Solution	A course staff believes this response is correct but not the optimal solution.
Green Highlighted Solution	A course staff believes this response is correct and is an optimal solution.

We have reduced the questions for Exam 5.

Questions **Removed**: 4, 7, 9, 10, 12, 13, 15, 16

Questions **Remaining**: 1, 2, 3, 5, 6, 8, 11, 14, 17, 18, 19

Graph Questions

No	Question	Ideas
----	----------	-------

1	A cycle is a sequence of vertices $v_1, v_2, \dots, v_k$ such that for each i, (v_i, v_{i+1}) is an edge, and (v_k, v_1) is an edge. Determine if G has a cycle. Try using both BFS and DFS if S.	1) DFS 2) BFS: modified version of BFS ($O(V+E)$), check if a vertex is a descendant of itself in BFS tree ($O(1)$) Brute Force: For each v in the set of vertices in the graph, perform a BFS or DFS search, with and if we visit v again, the graph is a cycle. $O(V(V+E))$ DFS: For each unvisited node, perform DFS. If a neighboring node is part of the DFS in stack and is not the current node's predecessor, there must be a cycle in the graph. Runtime: $O(V+E)$ BFS: Main idea - If the graph is acyclic, we should never encounter a neighboring node that is already visited by BFS and is not the node's predecessor Runtime: $O(V+E)$ ^ Modifying BFS above The general idea is to change the BFS alg. To store (parent, node) pairs. We can use the property that if we visit a node that has been visited previously, but isn't the parent of the current node, we have a cycle. 1.) We need a queue of (parent, node) pairs and an array of bools for visited nodes. We also need to populate the array by the size of the graph. 2.) Base Case: Push a (Null, root) node into the queue. We'll treat the root node as having no parent. 3.) Iteration: While the queue isn't empty, pop the latest (parent, node) pair from the queue. - a.) Begin iterating through all neighbors of the latest node. - b.) If the neighbor hasn't been visited, set its corresponding position in the visited array to True. Append that neighbor to the queue so that its corresponding neighbors can be processed. - c.) Otherwise, if the node has already been visited, but isn't the parent, you have a cycle. 4.) End: If the queue loop finishes executing, there's no cycle detected.
---	--	--

2	Given edge e, determine if G has a cycle involving e.	We're assuming that u,v edge exists. We remove the edge (u,v) then we run BFS to get from v to u. If we are able to get to u, then we know a path exists and IF it exists, then that means that the assumption is correct and a cycle exists with edge being a part of it. ($O(V+E)$) To add on to above, we can simply perform one BFS search with u as source, skipping the edge (u,v). Then, we check the final predecessor array to see if there still exist a path from u to v. Runtime: $O(V+E)$
3	A directed graph is called a DAG if it contains no directed cycles. Determine if a graph is a DAG.	Brute Force: For each u in the set of vertices in the graph, perform a BFS or DFS search, with and if we visit u again, the graph is a cycle. $O(V(V+E))$ Cycle Checking with DFS: Main idea: A graph is not a DAG if it contains cycles. That means for every unvisited vertex searched by DFS in a DAG, the neighboring vertices must only be either unvisited, or fully visited and no longer part of the DFS stack. Perform a DFS search on the graph. A cycle exists if while performing DFS and we iterate through all the neighbors in a vertex, we encounter a neighboring vertex that is in the current DFS recursion stack. One way to do this is to track the "state" of each vertex in the DFS search process as unvisited, traversing, or finished. A vertex is in a "traversing" state if it is part of the DFS recursion stack. A vertex is "finished" if it was visited and no longer part of the DFS recursion stack. Thus, if while visiting a vertex, we see that one of its neighbors is in a "traversing" state, then a back edge exists, and therefore it has a cycle and cannot be a DAG. Runtime: $O(V+E)$
5	A vertex v is a cut or articulation vertex if its removal disconnects two other vertices. Equivalently, all paths (in G, before removal) from u to u_0 pass through v. Determine if v is a cut vertex. If so, find some pair (u, u_0) that got disconnected.	1) Initialize: Start by marking all vertices as unvisited. Set up an array to store the discovery times of visited vertices and a parent array to keep track of the DFS tree. 2) Perform DFS: - From an arbitrary starting vertex, run a DFS. Use three arrays: - <code>disc[]</code>: Stores the time when a vertex is first discovered. - <code>low[]</code>: The lowest discovery time reachable from the vertex. - <code>parent[]</code>: To store the parent vertices in the DFS tree. - Maintain a time variable to keep track of discovery times. yun 3) DFS Recursive Function: - For each vertex u visited from v, check the following: - If u is unvisited, mark it as visited. Set <code>disc[u]</code> to time and <code>low[u]</code> to time. Increment time. - For each adjacent vertex u_0 of u:

		- If u_0 is not visited, set $parent[u_0]$ to u, and continue DFS with u_0. - After returning from the recursive DFS call, update $low[u]$ to the minimum of $low[u]$ and $low[u_0]$. - Check if the subtree rooted at u_0 has a connection back to one of the ancestors of u via a back-edge. If $low[u_0]$ is greater than $disc[u]$, then u is an articulation point. - If u_0 is visited and is not the parent of u, update $low[u]$ to the minimum of $low[u]$ and $disc[u_0]$. 4) Identify Cut Vertex: - During the DFS, if any vertex u satisfies the condition $low[u_0] > disc[u]$ for any adjacent u_0, then u is a cut vertex. 5) Finding Disconnected Pair: - If u is identified as a cut vertex and $low[u_0] > disc[u]$, then the pair (u, u_0) is a disconnected pair after the removal of vertex v. Brute Force: Main idea: Assuming the graph is connected and undirected, and vertex v is given, v is an articulation vertex if after removing v, doing a BFS or DFS search on any vertex u leaves unvisited vertices u'. The disconnected pair is simply (u, u'). - Implement a slightly modified BFS/DFS which skips any edges involving v - Pick any vertex u to perform the modified BFS/DFS with that is not vertex v - Check to see if there are any unvisited vertices (that is not v) - If there are no unvisited vertices, v is not an articulation vertex - Else return the aforementioned pair (u, u') (u' is simply any remaining unvisited vertex) Runtime: $O(V+E)$
6	The proof of Dijkstra's algorithm fails when there are negative edges. Why?	Dijkstra's algorithm assumes that the path to a node, once directly visited by Dijkstra's will contain the optimal distance from the source to that node. If there are negative weights, that means that somewhere in the undiscovered paths exists a new optimal distance to that node. This cannot happen with positive values, as future paths to that node will only be $\geq$ to the existing distance. Runtime for Dijkstra's algorithm: $O(E+V^2)=O(V^2)$
8	Given a directed (unweighted) graph G , the reverse is obtained by simply reversing all edges. Assume G is represented as an adjacency	Idea: To construct the reverse of a directed graph G, the direction of every edge in G needs to be reversed. Since G is an adjacency list of out neighbors, iterate over each vertex and its out neighbors to create an adjacency list for the reverse of directed graph G. For each edge in G $u \rightarrow v$, add an edge $v \rightarrow u$ in the reverse of G.

	list of out neighbors. Construct the reverse of G.	Runtime: $O(E)$
11	A common recommendation feature used in social networking websites is the number of common friends. In an undirected graph G, find a pair (u, v) with the largest number of common neighbors.	Brute Force: Loop through every possible pair (this means two nested loops we iterating through all vertices) and track the pair with the largest number of common neighbors $\text{len}(\text{intersection}(N(u), N(v)))$ Runtime: $O(V^2)$ Runtime: $O(V)$ Because you're going through every possible pair and not retracing your steps you're going through the graph once hence just V Runtime: $O(V^3)$ The time complexity to find intersection between two set of edges is $O(V)$, and if that occurs within the double loop, worst case is $O(V^3)$
14	Biologists often construct a food network of species in an ecosystem. The vertices represent species, and a directed edge (u, v) means species u eats species v. An apex species is one that is not eaten by another species. Suppose you are given a list of all possible "eating" relationships (so a list of "u eats v"). Determine all apex species.	Idea: Assuming the graph is stored has an adjacency list and directed The main idea is to use BFS search to mark each vertex has apex and non-apex. For each unvisited vertex v, mark v as "apex", then for each u in N(v), mark u as "non-apex", overriding any existing "apex" neighbors. Finally return all apex vertices by iterating through visited array. Runtime: $O(V+E)$ Different idea - using the given list of all (u,v). Initialize a list of all vertexes, iterate through the list of ordered pairs, if a vertex is the second element in the ordered pair (meaning it gets eaten by species u), remove it from the list of vertexes. After iterating through the whole list of ordered pairs, the list of vertexes should only contain apex species $O(E)$ Edit: $O(V+E)$
17	You are given a directed, unweighted graph G representing the power grid, as an adjacency list. Each vertex is a power station, and an edge (u, v) means that power goes from u to v. A source vertex denotes a power generator. A sink vertex denotes a power supplier,	Idea: Do a special DFS search on source s, marking nodes visited with a special marker in the visited array. Then do a regular DFS search for all remaining unvisited vertices that does not have s as a descendant [i.e. cancel DFS if we encounter vertex s]. If we encounter any special vertex marked by DFS(s), override the mark with a regular visited mark and continue DFS. Finally, return all sink vertices still uniquely marked by DFS(s) [i.e. just iterate through the visited array]. Runtime: $O(V+E)$

	typically to a neighborhood. Your job is to understand the robustness of the power grid, under failure of a power station s. (Note that this may not be a source, it could be some internal vertex.) If s fails, it stops transmitting power to any of the outneighbors. This could potentially result in some sink node t not receiving power at all, representing a power failure in a neighborhood. Thus, a sink node t will not receive power if every path from every source to t passes through the power station s. Design an algorithm that given G and a vertex s, determines all the sink vertices t (if any) that stop receiving power when s fails.	
18	A directed graph G is semiconnected if for all pairs of vertices u, v, there is either a path from u to v, or from v to u (but not both). Give an algorithm to determine if G is semiconnected.	The graph is semi-connected if and only if it has only one topological sort list. Thus, if it has only one topological sort list, it must be semi-connected. Determining whether G only has one topological sort list: Create topological sort of G. Find the longest path from the first vertex in the topological sort list to the last vertex in the topological sort list. If the longest path distance exists and is equal to the # of vertices in the graph - 1, there must only be one topological sort list. Also, the graph is not semi-connected if it contains a cycle, but if it contains cycle, the graph does not have a topological sorting. Runtime: $O(V+E)$
19	The diameter of a graph G is the largest shortest path distance in G (meaning, it is $\max_{u,v \in V} \text{dist}(u,v)$, where $\text{dist}(u,v)$ is the shortest path distance). Give an algorithm to compute the diameter of an undirected graph G. Give a more efficient algorithm to	Brute Force: Assuming graph is unweighted and connected For each vertex (s) in the graph, get the predecessor array from BFS(s), and find the vertex (s') with longest distance from s to s'. Repeat until every vertex's BFS tree is computed, and return maximum of the longest shortest paths Runtime: $O(V \cdot E + V^2 \cdot E)$ *Runtime: $O(V+E)$ - For each vertex v in G - Run BFS $O(V)$ - For each vertex u in G not v $O(V+E)$

	determine the diameter of an undirected tree T.	- Find the shortest path from (v,u) $O(E)$ For a general graph using the guidelines to the left. Begin by initializing all the integer variables needed to calculate the max shortest distance. For each vertex of the graph, perform a BFS starting with 's'. Identify the vertex that has the maximum distance from 's'. Continual loop, if the distance is greater than the current value of maxDist then update it. Otherwise return maxDist to get the diameter. Runtime: $O(V * (V + E))$ ^^this is the same as the highlight in red so I am not sure if they were trying to do the same thing?
--	--	--